Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования

«Алтайский государственный технический университет им. И. И. Ползунова»

Факультет информационных технологий Кафедра информатики, вычислительной техники и информационной безопасности

Отчет защищен с оценкой_____
Преподаватель _____С. В. Умбетов
«____»_____2026 г.

Отчёт по лабораторной работе №1 по дисциплине «Устройство и функционирование интеллектуальных информационных систем» «Лабораторная работа с табличными данными в Pandas»

ЛР 09.02.08.23.001

Студент группы 1ИИС-41 М.А.Синтиков
группа и.о., фамилия
Преподаватель старший преподаватель, доцент С. В. Умбетов
должность, ученая степень и.о., фамилия

Барнаул 2026

Лабораторная работа №1
Лабораторная работа с табличными данными в Pandas

Цель работы: Освоить базовые и продвинутые приёмы работы с табличными данными в библиотеке Pandas на примере реального набора данных о пассажирах «Титаника». Научиться загружать, исследовать, фильтровать, агрегировать и преобразовывать табличные данные, а также делать аналитические выводы на основе полученных результатов.

Задачи работы: В ходе выполнения лабораторной работы необходимо:
1. Изучить теоретические основы работы с библиотекой Pandas: объекты Series и DataFrame , их создание и основные свойства.
2. Освоить чтение данных из CSV-файлов с помощью функции pd.read_csv() и её параметров.
3. Научиться получать первичную информацию о датафрейме с помощью методов .head() , .tail() , .info() , .describe() .
4. Освоить индексацию и доступ к данным через .loc , .iloc и обращение к столбцам.
5. Научиться фильтровать данные по одному и нескольким условиям с использованием логических операторов.
6. Освоить статистические методы (.mean() , .max() , .min() , .sum() , .std() , .count()) и их применение к датафреймам и отдельным столбцам.
7. Изучить группировку данных с помощью .groupby() и агрегацию через .agg() .
8. Научиться строить сводные таблицы с помощью .pivot_table() .
9. Освоить работу с пропущенными значениями: обнаружение, удаление, заполнение.
10. Научиться создавать новые признаки на основе существующих с использованием арифметических операций, pd.cut() и .apply() .
11. Провести комплексный аналитический анализ датасета Titanic и сформулировать содержательные выводы.
12. Закрепить полученные навыки ответами на контрольные вопросы.

Задание принято:
Ситников М.А.

Подпись

Ход работы

Задача 1. Загрузка и первичный осмотр данных

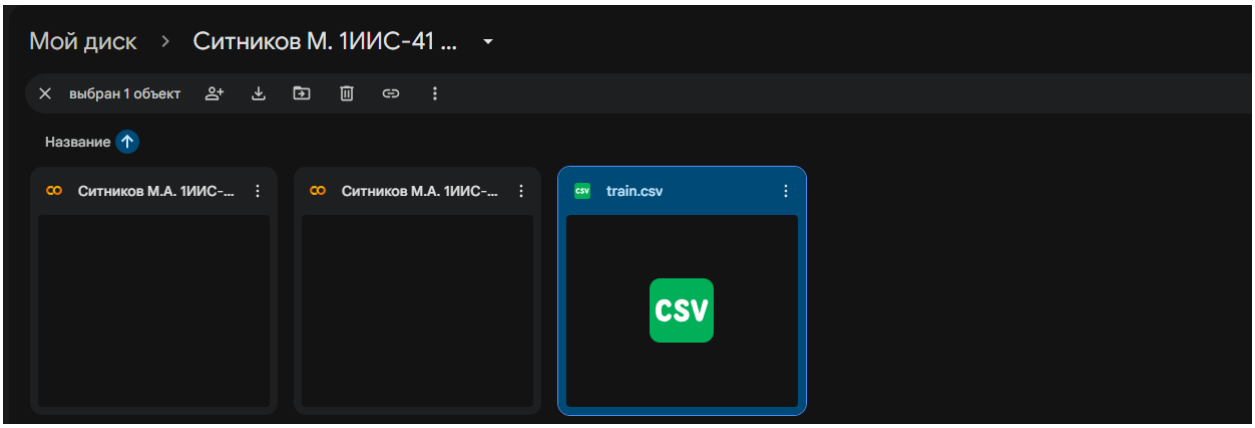
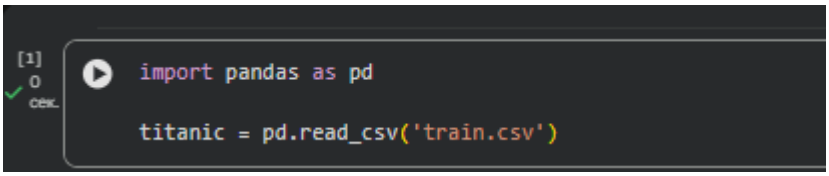


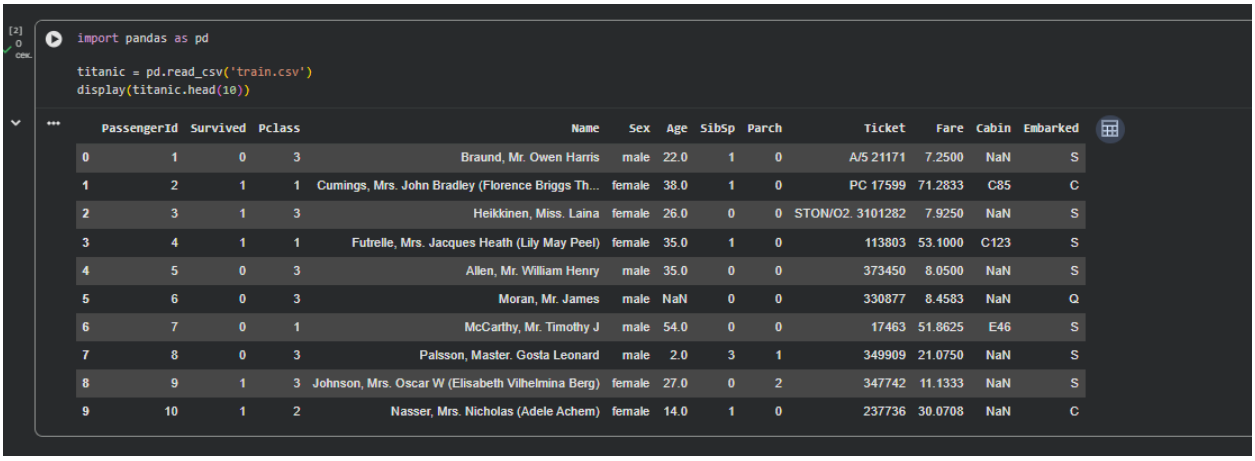
Рис.1 – Файл train.csv скачан и добавлен на гугл диск



```
import pandas as pd

titanic = pd.read_csv('train.csv')
```

Рис.2 – Данные train.csv загружены в DataFrame с помощью pd.read_csv()



```
import pandas as pd

titanic = pd.read_csv('train.csv')
display(titanic.head(10))
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

Рис.3 - Вывод первых 10 строк таблицы с помощью display(titanic.head(10))

```
[3]
0
сек.
import pandas as pd

titanic = pd.read_csv('train.csv')
display(titanic.head(10))
display(titanic.tail())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Рис.4 - Вывод последних 5 строк с помощью display(titanic.tail())

Задача 2. Получение информации о датафрейме

```
[4]
0
сек.
display(titanic.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

Рис-5 – Полученная информация о типах данных с помощью .info

Результат: .info() показало, что существует 2 столбца с дробными числами float64 (Age и Fare), 5 столбцов с целыми числами int64 (PassengerId, Survived, Pclass, SibSp и Parch) и 5 столбцов со сточными данными (Name, Sex, Ticket, Cabin, Embarked). Также в нескольких столбцах замечены

пропуски: Age (177 пропусков), Cabin (687 пропусков) и Embarked (2 пропуска)

```
[9]
0
сек.
display(titanic.info())
titanic.describe()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Рис.6 - .describe() применён к числовым столбцам.

```
display(titanic.info())
titanic.describe()
titanic.describe(include=['object'])
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3
top	Dooley, Mr. Patrick	male	347082	G6	S
freq	1	577	7	4	644

Рис.7 - .describe(include=['object']) применён к текстовым столбцам.

Результат: .describe(include=['object']) показал, что на корабле больше всего было мужчин (577 человек), а больше всего людей погрузилось с порта S (Саутгемптон)(644 человека)

Задача 3. Работа с Series

```
embarked_series = titanic['Embarked'].head(5).copy()
embarked_series.index = ['Первый', 'Второй', 'Третий', 'Четвёртый', 'Пятый']
display(embarked_series)
```

	Embarked
Первый	S
Второй	C
Третий	S
Четвёртый	S
Пятый	S

dtype: object

Рис.8 - Embarked преобразован в Series. Установлен произвольный индекс для первых 5 элементов этой Series.

```
[17]
embarked = titanic['Embarked'].head(5).copy()
embarked.index = ['Первый', 'Второй', 'Третий', 'Четвёртый', 'Пятый']
display(embarked)
embarked.loc[['Первый', 'Третий', 'Пятый']]
```

	Embarked
Первый	S
Второй	C
Третий	S
Четвёртый	S
Пятый	S

dtype: object

	Embarked
Первый	S
Третий	S
Пятый	S

dtype: object

Рис.9 – Получен доступ к первому, третьему и пятому элементу с помощью .loc

```
[19]
0
сек.
embarked = titanic['Embarked'].head(5).copy()
embarked.index = ['Первый', 'Второй', 'Третий', 'Четвёртый', 'Пятый']
display(embarked)
embarked.loc[['Первый', 'Третий', 'Пятый']]
embarked.iloc[[1, 3]]
```

```
...
            Embarked
Первый             S
Второй             C
Третий             S
Четвёртый          S
Пятый              S

dtype: object
            Embarked
Второй             C
Четвёртый          S

dtype: object
```

Рис.10 - Получен доступ ко второму и четвёртому элементу с помощью .iloc

Задача 4. Статистические методы

```
[4]
0
сек.
print('Средний возраст пасажиров:', titanic['Age'].mean())
print('Максимальная стоимость билета:', titanic['Fare'].max())
print('Минимальная стоимость билета:',titanic['Fare'].min())
print('Стандартное отклонение возраста:',titanic['Age'].std())
```

```
...
Средний возраст пасажиров: 29.69911764705882
Максимальная стоимость билета: 512.3292
Минимальная стоимость билета: 0.0
Стандартное отклонение возраста: 14.526497332334044
```

Рис.11 - Вычислен средний возраст пассажиров с помощью .mean() . Найдена максимальная и минимальная стоимость билета с помощью .max() и .min(). Вычислено стандартное отклонение для столбца с возрастом с помощью .std().

Результат:

Средний возраст пассажира ≈ 30 лет

Максимальная стоимость билета = 512.33

Минимальная стоимость билета = 0

Стандартное отклонение возраста ≈ 14.53 лет

Задача 5. Фильтрация данных по условиям

```
[5]
0
сек.
titanic[titanic['Age'] > 30]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S
...	...	...	...	...	...	...	...	...	...	...	...	...
873	874	0	3	Vander Cruyssen, Mr. Victor	male	47.0	0	0	345765	9.0000	NaN	S
879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50	C
881	882	0	3	Markun, Mr. Johann	male	33.0	0	0	349257	7.8958	NaN	S
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	NaN	Q
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

305 rows × 12 columns

Рис.12 – Пассажиры с возрастом более 30 лет отфильтрованы с помощью titanic[titanic[‘Age’] > 30].

Результат: Пассажиров старше 30 лет: 305 человек

```
[6]
0
сек.
titanic[titanic['Age'] > 30]
titanic[titanic['Fare'] > titanic['Fare'].mean()]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
23	24	1	1	Sloper, Mr. William Thompson	male	28.0	0	0	113788	35.5000	A6	S
27	28	0	1	Fortune, Mr. Charles Alexander	male	19.0	3	2	19950	263.0000	C23 C25 C27	S
...	...	...	...	...	...	...	...	...	...	...	...	...
856	857	1	1	Wick, Mrs. George Dennick (Mary Hitchcock)	female	45.0	1	1	36928	164.8667	NaN	S
863	864	0	3	Sage, Miss. Dorothy Edith "Dolly"	female	NaN	8	2	CA. 2343	69.5500	NaN	S
867	868	0	1	Roebling, Mr. Washington Augustus II	male	31.0	0	0	PC 17590	50.4958	A24	S
871	872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	11751	52.5542	D35	S
879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50	C

211 rows × 12 columns

Рис.13 – Пассажиры со стоимостью билета выше среднего отфильтрованы с помощью titanic[titanic[‘Fare’] > titanic[‘Fare’].mean()].

Результат: Пассажиров с билетом дороже среднего: 211 человек

```
[7]
0 сек.
titanic[titanic['Age'] > 30]
titanic[titanic['Fare'] > titanic['Fare'].mean()]
titanic[(titanic['Pclass']==1) & (titanic['Sex']=='female')]
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.5500	C103	S
31	32	1	1	Spencer, Mrs. William Augustus (Marie Eugenie)	female	NaN	1	0	PC 17569	146.5208	B78	C
52	53	1	1	Harper, Mrs. Henry Sleeper (Myna Haxtun)	female	49.0	1	0	PC 17572	76.7292	D33	C
...	...	...	...	...	...	...	...	...	...	...	...	...
856	857	1	1	Wick, Mrs. George Dennick (Mary Hitchcock)	female	45.0	1	1	36928	164.8667	NaN	S
862	863	1	1	Swift, Mrs. Frederick Joel (Margaret Welles Ba...	female	48.0	0	0	17466	25.9292	D17	S
871	872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	11751	52.5542	D35	S
879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.1583	C50	C
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S

94 rows × 12 columns

Рис.14 – Пассажиры женского пола из первого класса отфильтрованы с помощью titanic[(titanic[‘Pclass’] == 1) & (titanic[‘Sex’] == ‘female’)].

Результат: Женщин в первом классе: 94 человека

```
[10]
0 сек.
titanic[titanic['Age'] > 30]
titanic[titanic['Fare'] > titanic['Fare'].mean()]
titanic[(titanic['Pclass']==1) & (titanic['Sex']=='female')]
titanic[(titanic['Pclass']==1) & (titanic['Sex']=='female')]['Age'].mean()

np.float64(34.61176470588235)
```

Рис.15 –Вычислен средний возраст женщины из первого класса с помощью titanic[(titanic[‘Pclass’] == 1) & (titanic[‘Sex’] == ‘female’)][‘Age’].mean() .

Результат: Средний возраст женщины из первого класса ≈ 35 лет

Задача 6. Комбинированная фильтрация и статистика

```
survived = titanic[(titanic['Survived'] == 1) & (titanic['Age'] < 18)]
display(survived)
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.00	1	0	237736	30.0708	NaN	C
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.00	1	1	PP 9549	16.7000	G6	S
22	23	1	3	McGowan, Miss. Anna "Annie"	female	15.00	0	0	330923	8.0292	NaN	Q
39	40	1	3	Nicola-Yarred, Miss. Jamila	female	14.00	1	0	2651	11.2417	NaN	C
43	44	1	2	Laroche, Miss. Simonne Marie Anne Andree	female	3.00	1	2	SC/Paris 2123	41.5792	NaN	C
...	...	...	...	...	...	...	...	...	...	...	...	...
830	831	1	3	Yasbeck, Mrs. Antoni (Selini Alexander)	female	15.00	1	0	2659	14.4542	NaN	C
831	832	1	2	Richards, Master. George Sibley	male	0.83	1	1	29106	18.7500	NaN	S
853	854	1	1	Lines, Miss. Mary Conover	female	16.00	0	1	PC 17592	39.4000	D28	S
869	870	1	3	Johnson, Master. Harold Theodor	male	4.00	1	1	347742	11.1333	NaN	S
875	876	1	3	Najib, Miss. Adele Kiamie "Jane"	female	15.00	0	0	2667	7.2250	NaN	C

61 rows × 12 columns

Рис.16 – Найдены все пассажиры, которые выжили и возрастом менее 18 лет с помощью titanic[(titanic[‘Survived’] == 1) & (titanic[‘Age’] < 18)]

```
survived = titanic[(titanic['Survived'] == 1) & (titanic['Age'] < 18)]
display(survived)
print('Средняя стоимость билета выживших пассажиров:', survived['Fare'].mean())
print('Количестов выживших пассажиров:', survived['PassengerId'].count())
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.00	1	0	237736	30.0708	NaN	C
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.00	1	1	PP 9549	16.7000	G6	S
22	23	1	3	McGowan, Miss. Anna "Annie"	female	15.00	0	0	330923	8.0292	NaN	Q
39	40	1	3	Nicola-Yarred, Miss. Jamila	female	14.00	1	0	2651	11.2417	NaN	C
43	44	1	2	Laroche, Miss. Simonne Marie Anne Andree	female	3.00	1	2	SC/Paris 2123	41.5792	NaN	C
...	...	...	...	...	...	...	...	...	...	...	...	...
830	831	1	3	Yasbeck, Mrs. Antoni (Selini Alexander)	female	15.00	1	0	2659	14.4542	NaN	C
831	832	1	2	Richards, Master. George Sibley	male	0.83	1	1	29106	18.7500	NaN	S
853	854	1	1	Lines, Miss. Mary Conover	female	16.00	0	1	PC 17592	39.4000	D28	S
869	870	1	3	Johnson, Master. Harold Theodor	male	4.00	1	1	347742	11.1333	NaN	S
875	876	1	3	Najib, Miss. Adele Kiamie "Jane"	female	15.00	0	0	2667	7.2250	NaN	C

61 rows × 12 columns

```
Средняя стоимость билета выживших пассажиров: 34.857309836065575
Количестов выживших пассажиров: 61
```

Рис.17 – Вычислена средняя стоимость билета и количество выживших пассажиров с помощью survived[‘Fare’].mean() и survived[‘PassengerID’].count()

Результат:

Средняя стоимость билета выживших пассажиров ≈ 34.86

Количество выживших пассажиров = 61 человек

Задача 7. Группировка данных

```
[47]
0 сек.
titanic.groupby('Pclass')['Age'].mean()
```

Pclass	Age
1	38.233441
2	29.877630
3	25.140620

dtype: float64

Рис.18 – Вычислен средний возраст пассажиров для каждого из трёх классов с помощью titanic.groupby('Pclass')['Age'].mean()

Результат:
Средний возраст в первом классе ≈ 38 лет
Средний возраст во втором классе ≈ 30 лет
Средний возраст в третьем классе ≈ 25 лет

```
[48]
0 сек.
titanic.groupby('Pclass')['Age'].mean()
titanic.groupby('Pclass')['Fare'].agg(['count', 'mean'])
```

Pclass	count	mean
1	216	84.154687
2	184	20.662183
3	491	13.675550

Рис.19 – Вычислена средняя стоимость билета и количество пассажиров для каждого из трёх классов с помощью titanic.groupby('Pclass')['Fare'].agg(['count', 'mean'])

Результат:
Средняя цена билета в первом классе ≈ 84.15 Количество пассажиров = 216
Средняя цена билета во втором классе ≈ 20.66 Количество пассажиров = 184
Средняя цена билета в третьем классе ≈ 13.68 Количество пассажиров = 491

```
[54]
0 сек.
titanic.groupby('Pclass')['Age'].mean()
titanic.groupby('Pclass')['Fare'].agg(['count', 'mean'])
titanic.groupby('Pclass')['Fare'].agg(['count', 'mean']).sort_values('mean', ascending=False)
```

	count	mean
Pclass		
1	216	84.154687
2	184	20.662183
3	491	13.675550

Рис.20 – Результат отсортирован по средней стоимости билета в порядке убывания с помощью titanic.groupby('Pclass')['Fare'].agg(['count', 'mean']).sort_values('mean', ascending=False)

Задача 8. Сводная таблица

```
[55]
0 сек.
titanic.pivot_table(values='Survived', index='Pclass', columns='Sex', aggfunc='mean')
```

Sex	female	male
Pclass		
1	0.968085	0.368852
2	0.921053	0.157407
3	0.500000	0.135447

Рис.21 - Создана сводная таблица с помощью .pivot_table() , где: значения — доля выживших (Survived); строки — классы билета (Pclass); столбцы — пол (Sex).

Результат: Выше всего доля выживших у женщин первого класса (≈ 97%), ниже всего у мужчин третьего класса (≈ 14%).

Задача 9. Работа с пропущенными значениями

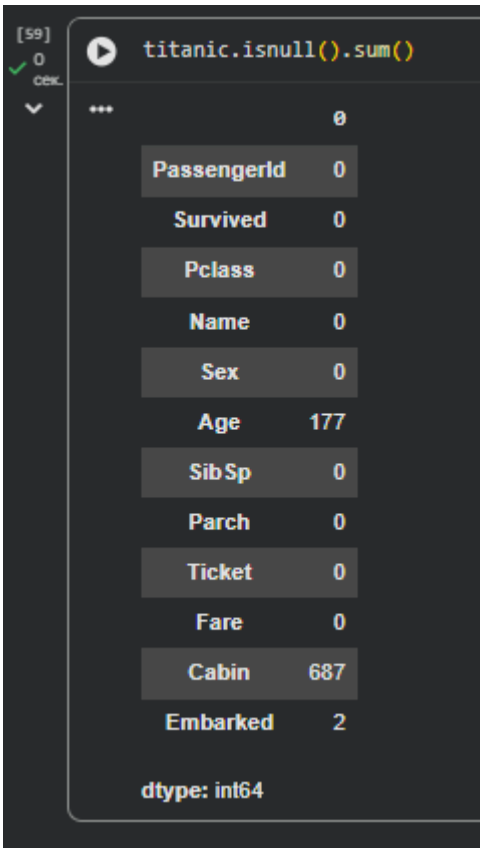


Рис.22 – Выведено количество пропусков с помощью titanic.isnull().sum()

Результат: В столбце Age пропущено 177 значений, в Cabin пропущено 687 значений, в Embarked пропущено 2 значения.

```
[56]
0 сек.
titanic.isnull().sum()
titanic.isnull().sum() / len(titanic) * 100
```

	0
PassengerId	0.000000
Survived	0.000000
Pclass	0.000000
Name	0.000000
Sex	0.000000
Age	19.865320
SibSp	0.000000
Parch	0.000000
Ticket	0.000000
Fare	0.000000
Cabin	77.104377
Embarked	0.224467

dtype: float64

Рис.23 – Выведен процент пропусков с помощью titanic.isnull().sum() / len(titanic) * 100

Результат: Процент пропущенных данных в Age ≈ 19.87%, в Cabin ≈ 77.1%, в Embarked ≈ 0.22%

```
[64]
0 сек.
titanic.isnull().sum()
titanic.isnull().sum() / len(titanic) * 100
titanic['Embarked'] = titanic['Embarked'].fillna('S')
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].mean())
```

Рис.24 – Пропуски в столбце Embarked заполнены значением S с помоью titanic['Embarked'] = titanic['Embarked'].fillna('S'). Пропуски в столбце Age заполнены медианой с помощью titanic['Age'] = titanic['Age'].fillna(titanic['Age'].mean()).

```
[65]
0
сек.
titanic.isnull().sum()
titanic.isnull().sum() / len(titanic) * 100
titanic['Embarked'] = titanic['Embarked'].fillna('S')
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].mean())
titanic.duplicated(subset=['PassengerId']).sum()

np.int64(0)
```

Рис.25 – Проверка на наличие дубликатов с помощью titanic.duplicated(subset=['PassengerId']).sum()

Результат: Дубликаты не найдены (np.int64(0))

```
[66]
0
сек.
titanic.isnull().sum()
titanic.isnull().sum() / len(titanic) * 100
titanic['Embarked'] = titanic['Embarked'].fillna('S')
titanic['Age'] = titanic['Age'].fillna(titanic['Age'].mean())
titanic.duplicated(subset=['PassengerId']).sum()
titanic.drop_duplicates(subset=['PassengerId'], keep='first')
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	28.0	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 12 columns

Рис.26 – Устранение дубликатов с помощью titanic.drop_duplicates(subset=['PassengerId'], keep='first')

Задача 10. Создание новых признаков и расширенный анализ

```
[68]
0
сек.
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
```

Рис.27 – Создан новый признак FamilySize, который равен SibSp+Parch+1 с помощью titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1

```
[69]
0
сек.
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
```

Рис.28 – Создан признак IsAlone, который равен 1 если FamilySize равен 1. В другом случае (если FamilySize >1) IsAlone равно 0.

```
[16]
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
titanic['AgeGroup'] = pd.cut(titanic['Age'], bins=[0, 12, 18, 35, 60, 100], labels=['ребёнок', 'подросток', 'молодой', 'взрослый', 'пожилой'])
```

Рис.29 – Создан признак AgeGroup с помощью pd.cut() (группы: ребёнок 0–12, подросток 13–18, молодой 19–35, взрослый 36–60, пожилой 60+).

```
[20]
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
titanic['AgeGroup'] = pd.cut(titanic['Age'], bins=[0, 12, 18, 35, 60, 100], labels=['ребёнок', 'подросток', 'молодой', 'взрослый', 'пожилой'])
titanic.groupby('AgeGroup', observed=True)['Survived'].mean()
```

AgeGroup	Survived
ребёнок	0.579710
подросток	0.428571
молодой	0.382682
взрослый	0.400000
пожилой	0.227273

dtype: float64

Рис.30 – Вычислена доля выживших с помощью titanic.groupby('AgeGroup', observed=True)[‘Survived’].mean()

Результат:

Самая большая доля выживших у детей (≈ 58%), а самая маленькая у стариков (≈ 23%)

```
[21]
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
titanic['AgeGroup'] = pd.cut(titanic['Age'], bins=[0, 12, 18, 35, 60, 100], labels=['ребёнок', 'подросток', 'молодой', 'взрослый', 'пожилой'])
titanic.groupby('AgeGroup', observed=True)['Survived'].mean()
titanic.groupby('AgeGroup', observed=True)['Fare'].mean()
```

AgeGroup	Fare
ребёнок	31.536352
подросток	33.451313
молодой	29.925929
взрослый	44.259658
пожилой	41.371214

dtype: float64

Рис.31 – Вычислена средняя стоимость билета с помощью titanic.groupby('AgeGroup', observed=True)[‘Fare’].mean()

Самая большая средняя стоимость билетов у взрослых (≈ 44.25), а самая маленькая у детей (≈ 31.53)

```
[22]
0
сек.
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
titanic['AgeGroup'] = pd.cut(titanic['Age'], bins=[0, 12, 18, 35, 60, 100], labels=['ребёнок', 'подросток', 'молодой', 'взрослый', 'пожилой'])
titanic.groupby('AgeGroup', observed=True)['Survived'].mean()
titanic.groupby('AgeGroup', observed=True)['Fare'].mean()
titanic.groupby('AgeGroup', observed=True)['PassengerId'].count()
```

AgeGroup	PassengerId
ребёнок	69
подросток	70
молодой	358
взрослый	195
пожилой	22

dtype: int64

Рис.32 – Вычислено количество пассажиров с помощью titanic.groupby('AgeGroup', observed=True)[‘PassengerId’].count()

Самая распространённая возрастная группа пассажиров – это молодые (358), а самая редкая – старики (22)

```
[23]
0
сек.
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1
titanic['IsAlone'] = titanic['FamilySize'].apply(lambda x: 1 if x == 1 else 0)
titanic['AgeGroup'] = pd.cut(titanic['Age'], bins=[0, 12, 18, 35, 60, 100], labels=['ребёнок', 'подросток', 'молодой', 'взрослый', 'пожилой'])
titanic.groupby('AgeGroup', observed=True)['Survived'].mean()
titanic.groupby('AgeGroup', observed=True)['Fare'].mean()
titanic.groupby('AgeGroup', observed=True)['PassengerId'].count()
titanic.pivot_table(values='Survived', index='AgeGroup', columns='Sex', aggfunc='mean', observed=True)
```

Sex	female	male
AgeGroup		
ребёнок	0.593750	0.567568
подросток	0.750000	0.088235
молодой	0.783333	0.180672
взрослый	0.771429	0.192000
пожилой	1.000000	0.105263

Рис.33 – Построена сводная таблица по возрастным группам с помощью titanic.pivot_table(values='Survived', index='AgeGroup', columns='Sex', aggfunc='mean', observed=True)

Результат: Все пожилые женщины выжили (100%), самая низкая доля выживших у подростков мужского пола (≈ 9%).

Задача 11. Комплексный аналитический отчёт

```
[73]
0
сек.
titanic['Embarked'].value_counts().head(3)
```

Embarked	count
S	646
C	168
Q	77

dtype: int64

Рис.34 – Определены три порта посадки по количеству пассажиров с помощью titanic['Embarked'].value_counts().head(3)

Результат: больше всего людей погрузилось с порта S(Саутгемптон), меньше всего с порта Q(Куинстаун)

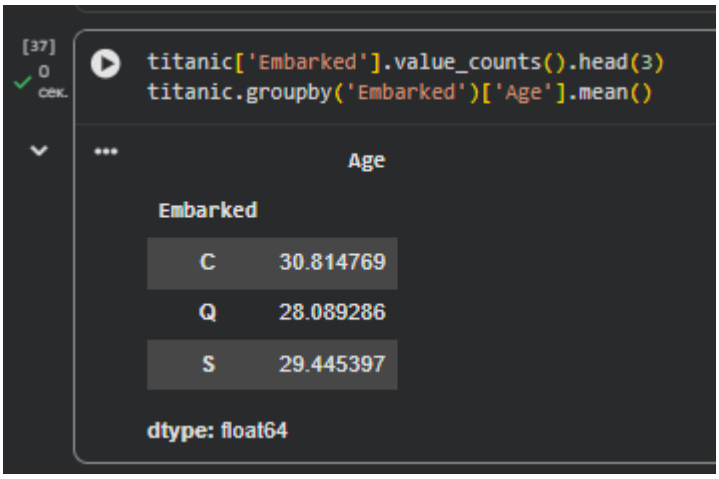


Рис.35 – Для каждого порта вычислен средний возраст пассажиров

Результат:
Средний возраст в порту C(Шербур) ≈ 31 год
Средний возраст в порту Q(Куинстаун) ≈ 28 год
Средний возраст в порту S(Саутгемптон)≈ 29 год

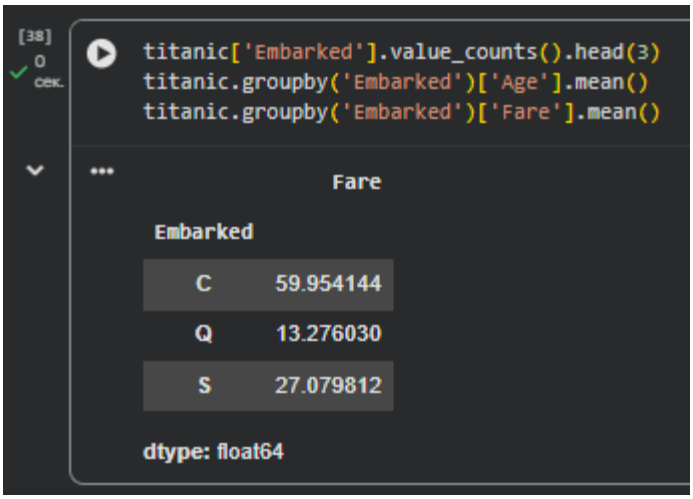


Рис.36 – Для каждого порта вычислена стоимость билета

Результат:
Самые дорогие билеты были в порту C(Шербур) ≈ 60
Самые дешёвые билеты были в порту Q(Куинстаун) ≈ 13.2

```
[40]
0 сек.
titanic['Embarked'].value_counts().head(3)
titanic.groupby('Embarked')['Age'].mean()
titanic.groupby('Embarked')['Fare'].mean()
titanic.groupby('Embarked')['Survived'].mean()
```

	Survived
Embarked	
C	0.553571
Q	0.389610
S	0.336957

dtype: float64

Рис.37 – Для каждого порта вычислена доля выживших пассажиров

Результат: Наибольшее количество выживших из порта C(Шербур) (≈ 55%), меньше всего выжило пассажиров с порта S(Саутгемптон)(≈ 33%)

```
[42]
0 сек.
titanic['Embarked'].value_counts().head(3)
titanic.groupby('Embarked')['Age'].mean()
titanic.groupby('Embarked')['Fare'].mean()
titanic.groupby('Embarked')['Survived'].mean()
titanic.groupby('Embarked')['Pclass'].apply(lambda x: (x == 1).mean())
```

	Pclass
Embarked	
C	0.505952
Q	0.025974
S	0.197205

dtype: float64

Рис.38 – Для каждого порта вычислена доля пассажиров из первого класса

Результат: Больше всего выжило пассажиров первого класса из порта C(Шербур) (≈ 51%), меньше всего из порта Q(Куинстаун) (≈ 3%)

В результате я выяснил, что порт C(Шербур) связан с наибольшей долей выживших (≈ 55%), так как большая часть пассажиров была из первого класса (≈ 51%) и они, скорее всего, имели приоритет при эвакуации с корабля.

Задача 12. Сравнительный анализ по размеру семьи

```
[43]
0 сек.
titanic.groupby('FamilySize')['Survived'].mean()
```

FamilySize	Survived
1	0.303538
2	0.552795
3	0.578431
4	0.724138
5	0.200000
6	0.136364
7	0.333333
8	0.000000
11	0.000000

dtype: float64

Рис.39 – Данные FamilySize отсортированы по доле выживших

Результат: Чаще всего выживали семьи из 4-х человек (≈72%). Семьи с числом пассажиров 8 и больше человек не выживали.

```
[44]
0 сек.
titanic.groupby('FamilySize')['Survived'].mean()
titanic.groupby('FamilySize')['Age'].mean()
```

FamilySize	Age
1	32.220297
2	31.391511
3	26.035806
4	18.274815
5	20.818182
6	18.409091
7	15.166667
8	15.666667
11	NaN

dtype: float64

Рис.40 – Данные FamilySize отсортированы по среднему возрасту

Результат: Самый большой возраст в среднем был у одиночек (≈ 32 года). Самый маленький был в семьях с 7 пассажирами (≈ 15 лет)

```
titanic.groupby('FamilySize')['Survived'].mean()
titanic.groupby('FamilySize')['Age'].mean()
titanic.groupby('FamilySize')['Fare'].mean()
```

FamilySize	Fare
1	21.242689
2	49.894129
3	39.692482
4	54.864510
5	58.094453
6	73.722727
7	29.366667
8	46.900000
11	69.550000

dtype: float64

Рис.41 – Данные FamilySize отсортированы по средней стоимости билета

Результат: Самые дорогие билеты в среднем были у семей с 6 пассажирами (≈ 74), самые дешёвые были у одиночек (≈ 21)

```
[46]
0 сек.
titanic.groupby('FamilySize')['Survived'].mean()
titanic.groupby('FamilySize')['Age'].mean()
titanic.groupby('FamilySize')['Fare'].mean()
titanic.groupby('FamilySize')['PassengerId'].count()
```

FamilySize	PassengerId
1	537
2	161
3	102
4	29
5	15
6	22
7	12
8	6
11	7

dtype: int64

Рис.42 – Данные FamilySize отсортированы по количеству пассажиров

Результат: Больше всего на борту было одиночек (537), меньше всего было семей с 8 пассажирами (6)

```
[48]
0 сек.
titanic.groupby('FamilySize')['Survived'].mean()
titanic.groupby('FamilySize')['Age'].mean()
titanic.groupby('FamilySize')['Fare'].mean()
titanic.groupby('FamilySize')['PassengerId'].count()
big_families = titanic[titanic['FamilySize'] > 10]
display(big_families[['FamilySize', 'Survived']])
```

	FamilySize	Survived
159	11	0
180	11	0
201	11	0
324	11	0
792	11	0
846	11	0
863	11	0

Рис.43 – Семьи, в которых пассажиров больше 10 отфильтрованы командой big_families = titanic[titanic['FamilySize'] > 10]

Результат: Ни одна семья с количеством пассажиров > 10 не выжила

```
[49]
0
сек.
titanic.groupby('FamilySize')['Survived'].mean()
titanic.groupby('FamilySize')['Age'].mean()
titanic.groupby('FamilySize')['Fare'].mean()
titanic.groupby('FamilySize')['PassengerId'].count()
big_families = titanic[titanic['FamilySize'] > 10]
display(big_families[['FamilySize', 'Survived']])
family_stats = titanic.groupby('FamilySize')['Survived'].agg('mean').sort_values(ascending=False)
display(family_stats)
```

	FamilySize	Survived
159	11	0
180	11	0
201	11	0
324	11	0
792	11	0
846	11	0
863	11	0

FamilySize	Survived
4	0.724138
3	0.578431
2	0.552795
7	0.333333
1	0.303538
5	0.200000
6	0.136364
8	0.000000
11	0.000000

Рис.44 – Данные отсортированы по доле выживших

```
[58]
0
сек.
titanic.groupby('FamilySize')['Survived'].mean()
titanic.groupby('FamilySize')['Age'].mean()
titanic.groupby('FamilySize')['Fare'].mean()
titanic.groupby('FamilySize')['PassengerId'].count()
big_families = titanic[titanic['FamilySize'] > 10]
display(big_families[['FamilySize', 'Survived']])
family_stats = titanic.groupby('FamilySize')['Survived'].agg('mean').sort_values(ascending=False)
display(family_stats)
print('Одиночки:', titanic[titanic['FamilySize'] == 1]['Survived'].mean())
print('Маленькие:', titanic[(titanic['FamilySize'] >= 2) & (titanic['FamilySize'] <= 4)]['Survived'].mean())
print('Большие:', titanic[titanic['FamilySize'] >= 5]['Survived'].mean())
```

	FamilySize	Survived
159	11	0
180	11	0
201	11	0
324	11	0
792	11	0
846	11	0
863	11	0

FamilySize	Survived
4	0.724138
3	0.578431
2	0.552795
7	0.333333
1	0.303538
5	0.200000
6	0.136364
8	0.000000
11	0.000000

```
dtype: float64
Одиночки: 0.30353817504655495
Маленькие: 0.5787671232876712
Большие: 0.16129032258064516
```

Рис.45 – Сравнение доли выживших среди разных семей(одиночки, маленькие и большие)

Результат: Максимальная доля выживших у маленьких семей (≈58%). Меньше всего выжило больших семей (≈16%)

Ссылка на GitHub:
https://github.com/Enot2130/Sitnikov_1iis41/tree/main/Устройство%20и%20функционирование%20интеллектуальных%20информационных%20систем/Лабораторная%20работа%20с%20табличными%20данными%20в%20Pandas

Ссылка на Google Colab:

https://colab.research.google.com/drive/1ZtHcKNx0iiOZWGZqaxucAnGe9ZKxdMSB?usp=sharing

Контрольные вопросы

1. Что такое Pandas и для каких задач используется эта библиотека?
Pandas — библиотека Python для работы с табличными данными. Используется для загрузки, фильтрации, агрегации, преобразования данных и анализа.

2. Чем Series отличается от обычного списка Python? Как создать Series с пользовательским индексом?
Series — одномерный массив с индексами, которые могут быть любыми метками, а в списке — только порядковые номера. Создание: pd.Series(data, index=['a', 'b', 'c']).

3. В чём разница между .loc и .iloc при доступе к элементам Series?
.loc — доступ по меткам индекса (data.loc['a']). .iloc — по порядковым номерам (data.iloc[0]).

4. Как создать DataFrame из словаря? Как задать названия колонок и индекс вручную?
Из словаря: pd.DataFrame({'col1': [1, 2], 'col2': [3, 4]}). Вручную: pd.DataFrame([[1, 2], [3, 4]], columns=['foo', 'bar', 'baz'], index=['foobar', 'foobaz']).

5. Какие параметры функции pd.read_csv() вы знаете? Для чего используется параметр usecols?
sep — разделитель данных, по умолчанию ',' ;
decimal — разделитель числа на целую и дробную часть, по умолчанию '.' ;
names — список с названиями колонок, не обязательный параметр;
skiprows — если файл содержит системную информацию, можно просто её пропустить. Не обязательный параметр;
usecols — позволяет загрузить только определённые столбцы, что ускоряет чтение и экономит память;
index_col — устанавливает столбец в качестве индекса;
parse_dates — автоматически преобразует столбцы с датами в формат datetime.

6. Какую информацию выводит метод .info()? Какие параметры можно ему передать?
.info() выводит индекс, количество и названия колонок, непустые значения, типы данных и память. Параметры:
verbose: печатать ли информацию о DataFrame полностью;
memory_usage: печатать ли потребление памяти (True / False / 'deep');
null_counts: подсчитывать ли количество пустых элементов (по умолчанию True).

7. Что показывает метод .describe() для числовых данных и для текстовых данных?

Метод describe показывает основные статистические характеристики данных. Для числовых: число непустых значений (count), среднее (mean), стандартное отклонение (std), минимум и максимум (min и max), 25% и 75% квартили и медиана (50%). Для текстовых (include=['object']): выводится количество непустых строк, уникальных значений, самое частое значение (top) и частота этого значения (freq).

8. Как получить доступ к отдельному столбцу DataFrame? Какие два способа существуют?

Два способа: df['column_name'] или df.column_name.

9. Как отфильтровать DataFrame по одному условию? Как комбинировать несколько условий?

Одно условие: df[df[‘Age’] > 30]. Несколько: через & (И) и | (ИЛИ), условия в скобках: df[(df[‘Age’] > 30) & (df[‘Pclass’] == 1)].

10. Какие статистические методы можно применять к DataFrame и Series? Что произойдёт, если применить .mean() ко всему DataFrame с текстовыми столбцами?

.max() - максимум, .min()-минимум, .mean()-среднее значение, .sum()-сумма, .count()-количество не пустых строк, .std()-среднее отклонение. Если применить .mean() ко всему DataFrame с текстом, текстовые столбцы игнорируются (или вызовут ошибку без numeric_only=True).

11. Для чего используется метод .groupby()? Как применить несколько агрегирующих функций одновременно?

.groupby() группирует данные по столбцу для агрегации. Несколько функций: .agg(['mean', 'max', 'count']).

12. Чем сводная таблица (pivot_table) отличается от обычной группировки? В каких случаях удобнее использовать каждую из них?

pivot_table — это группировка с представлением результата в виде таблицы. Удобнее, когда нужно сравнить значения по двум признакам сразу. Обычная группировка проще для одной оси.

13. Как обнаружить и обработать пропущенные значения в DataFrame? Какие стратегии заполнения пропусков вы знаете?

Обнаружение: df.isnull().sum(). Удаление: dropna(). Заполнение: fillna(значение) — средним, медианой, модой или константой.

14. Как создать новый признак на основе существующих? Для чего используется метод pd.cut()?

Новый признак: df['FamilySize'] = df['SibSp'] + df['Parch'] + 1. pd.cut() — категоризация числовых данных по интервалам: pd.cut(df['Age'], bins=[0, 12, 18, 60, 100], labels=['ребёнок', 'подросток', 'взрослый', 'пожилой']).

15. Как отсортировать DataFrame по значениям одного или нескольких столбцов? Как вывести только топ-N строк?

Сортировка: df.sort_values('Age') или по нескольким: df.sort_values(['Pclass', 'Age']). Топ-N: df.sort_values('Fare', ascending=False).head(N).

Вывод

В ходе лабораторной работы я познакомился с библиотекой Pandas и освоил базовые приёмы работы с табличными данными на примере датасета о пассажирах Титаника. Начал с загрузки данных через pd.read_csv() и первичного осмотра таблицы — это помогло понять структуру DataFrame, типы данных и наличие пропусков.

При выполнении заданий я отработал создание Series и DataFrame, доступ к элементам через .loc и .iloc, фильтрацию по одному и нескольким условиям, статистические методы, группировку через .groupby() и .agg(), построение сводных таблиц через .pivot_table(), а также работу с пропусками и создание новых признаков с помощью pd.cut() и .apply().

Таким образом, лабораторная работа позволила освоить ключевые инструменты Pandas для загрузки, исследования, фильтрации и агрегации данных, а также научиться делать аналитические выводы на основе статистики.